Weibull RMST: mathematics and numerical choices

30 October 2026

1 Scope and conclusions

This report explains `_weibull_rmst` in `src/survjax/_numerics.py` at commit `b334074`. The public caller is `Weibull.rmst`. No implementation or specification change is proposed by writing this report.

The algorithm uses one exact identity and two controlled approximations:

Region, with $x = (\tau/b)^k$	Returned expression	Reason
$x < 0.1$	Integrated exponential series through degree 12	Avoid loss of a tiny power or gamma probability
$0.1 \le x \le 64 + 4/k$	Incomplete-gamma identity in log arithmetic	Use the available closed form
$x > 64 + 4/k$	The unrestricted mathematical mean	The omitted gamma tail is negligible

The series has **13 terms including its constant term**, evaluated by 12 loop iterations. Its real-arithmetic relative truncation error is below 1.78×10^{-23}. The mean-tail approximation has relative error below 1.43×10^{-21}. These bounds concern approximations, not floating-point rounding, the accuracy of JAX special functions, or all parameter derivatives.

There is an approval question at the zero boundary: the current branch has the specified value and first derivatives, but can give an incorrect higher right cutoff derivative. Section 7 makes this concrete; D-028 records it as open.

2 Exact integral and gamma identity

For positive shape k, positive scale b, and finite cutoff $\tau > 0$, write

$$S(t) = \exp\{-(t/b)^k\}, \qquad M(k, b, \tau) = \int_0^\tau S(t)\, dt.$$

Set $a = 1/k$ and substitute $u = (t/b)^k$. Then $dt = ba\, u^{a-1} du$ and

$$M = ba \int_0^x u^{a-1} e^{-u}\, du = ba\, \gamma(a, x) = b\Gamma(1 + a) P(a, x).$$

Here γ is the lower incomplete gamma function, $P(a, x) = \gamma(a, x)/\Gamma(a)$, and $Q(a, x) = 1 - P(a, x)$. These definitions are given in NIST DLMF section 8.2. The unrestricted mathematical mean is $\mu = b\Gamma(1 + a)$, so $M = \mu(1 - Q)$. This does **not** implement the public `mean()` method, which remains unavailable.

An equivalent normalization, useful for the lower tail, is

$$M = \tau \int_0^1 \exp(-xv^k)\, dv.$$

The target is always between 0 and τ. A tiny x therefore means M is close to τ, not close to zero.

3 Lower tail: integrated exponential series

Expanding the last integrand and integrating each term gives

$$M = \tau \sum_{n=0}^{\infty} \frac{(-x)^n}{n!(kn+1)}.$$

Termwise integration is valid by uniform absolute convergence on $v \in [0,1]$ for fixed finite x. This is also consistent with the incomplete-gamma series in NIST DLMF section 8.7. The implementation evaluates

$$M_{12} = \tau \sum_{n=0}^{12} \frac{(-x)^n}{n!(kn+1)}.$$

It starts `term = 1`, updates `term *= -x/n`, and adds `term/(k*n+1)`. It forms neither factorials nor large powers explicitly.

For $0 \le x < 0.1$, the magnitudes decrease: the ratio of consecutive terms is

$$\frac{x}{n+1}\frac{kn+1}{k(n+1)+1} < \frac{0.1}{n+1}.$$

The alternating-series remainder therefore satisfies

$$|M - M_{12}| \le \tau \frac{x^{13}}{13!(13k+1)}.$$

Since $M \ge \tau e^{-x}$, a uniform relative bound is

$$\frac{|M - M_{12}|}{M} \le e^x \frac{x^{13}}{13!(13k+1)} < e^{0.1}\frac{0.1^{13}}{13!} < 1.78 \times 10^{-23}.$$

Thus `0.1` makes convergence rapid and the fixed loop simple; degree 12 gives ample margin below float64 rounding. These choices are conservative, not claimed to be the smallest sufficient degree or an optimized crossover.

The Python `for` is JAX-compatible here because its trip count is fixed: `jit` unrolls the 12 iterations while tracing; array arithmetic then executes in the compiled program. It is not a data-dependent Python stopping rule. See the JAX scan documentation. A static `lax.fori_loop` or `lax.scan` could instead keep the recurrence rolled and reduce graph size; a speed advantage has not been measured. The current `jit`/`vmap`/derivative tests establish compatibility, not optimal performance.

The degree and crossover should be treated as documented internal numerical choices, preferably with named constants if the implementation is refined. The current source gives their literals and remainder comment; this report makes their coupling explicit. Degree 12 is sufficient, not uniquely required. I would not expose a raw term-count knob: a smaller count could silently break the accuracy guarantee. A future configurable summary API should express an accuracy/work budget, with a verified backend choosing terms. Staged section 8.7 explicitly provides no `options` argument, so no such API is added here.

Differentiating the polynomial with respect to the cutoff also has a useful form:

$$\frac{\partial M_{12}}{\partial \tau} = \sum_{n=0}^{12} \frac{(-x)^n}{n!}.$$

Its absolute error as an approximation to $S(\tau) = e^{-x}$ is at most $x^{13}/13! < 1.61 \times 10^{-23}$. This is a cutoff-derivative bound only; it is not a uniform bound for every shape, scale, or higher derivative. If computing x underflows to zero, the polynomial returns τ naturally. No epsilon, probability floor, or altered cutoff is introduced.

4 Middle: the identity, evaluated in log arithmetic

The code first computes $\ell = k(\log \tau - \log b) = \log x$, without forming τ/b. For the middle branch it evaluates `gammainc(a, exp(ell))` and

$$\log M = \log b + \log \Gamma(1 + a) + \log P(a, x).$$

Only the final result is exponentiated. This avoids an overflowing mean factor multiplied by a small probability when the final product is representable. It does not implement a new incomplete-gamma algorithm: special-function accuracy still depends on JAX.

On the specified shape range, $a \in [0.02, 5]$, and the middle branch has $x \in [0.1, 84]$. In particular, the gamma probability is not in an extreme underflow regime there. Outside the promised range, `gammainc` can return zero for a genuinely positive probability. The code substitutes a safe operand before taking its logarithm, then returns NaN for that result. It does not turn lost precision into a plausible zero RMST. For example, the independent review found float32 NaNs at $k = 0.02$, $b = 1$, $\tau = 1$; this is outside the promised shape range, not a new parameter-domain restriction.

5 Upper tail: why the mean is accurate enough

Returning μ instead of M has absolute error $\mu Q(a, x)$ and relative error

$$\frac{\mu - M}{M} = \frac{Q(a, x)}{1 - Q(a, x)}.$$

A direct bound explains both constants in the threshold. Let Y have a gamma law with shape a and unit scale. From its defining integral, for $0 < s < 1$,

$$\mathrm{E}[e^{sY}] = (1 - s)^{-a}.$$

Markov's inequality with $s = 3/4$ gives

$$Q(a, x) = \Pr(Y \geq x) \leq e^{-3x/4} 4^a.$$

If $x > 64 + 4a$, this becomes

$$Q(a, x) < \exp\{-48 + a(\log 4 - 3)\} < e^{-48} = 1.4251641 \times 10^{-21},$$

because $\log 4 - 3 < 0$ and $a > 0$. Consequently the relative error is below $e^{-48}/(1 - e^{-48})$. This real-arithmetic bound holds for every positive k; it does not guarantee successful floating-point evaluation outside the tested range. The fixed margin 64 is deliberately generous; the $4a$ term controls dependence on gamma shape. Neither constant was selected by loosening a test tolerance.

The cutoff derivative of this branch is zero, whereas the exact derivative is $S(\tau) = e^{-x} < e^{-64} < 1.61 \times 10^{-28}$. That is negligible at the required absolute scale 1 for cutoff derivatives, but it is **not relative accuracy** for an extremely small nonzero derivative. Parameter derivatives of the mean also approximate those of RMST; the value bound alone does not prove their errors.

The threshold is tested as $\ell > \log(64 + 4/k)$, using `logaddexp` to form the right side. The large, possibly unrepresentable x need never be constructed.

6 Branches, zero, invalid inputs, and floating point

The thresholds depend on x, not on τ/b alone. For illustration:

k	Series when τ/b is below	Mean when τ/b is above
0.2	10^{-5}	$4.182119424 \times 10^{9}$
1	0.1	68
50	0.9549925860	1.0867620143

Within the promised ranges, $k = 50$ makes x run from 10^{-100} to 10^{250}. Both endpoints are representable in float64, but not in float32. The explicit stress tests go further: $\tau/b = 10^{-8}$ and 10^{8} give $x = 10^{-400}$ and 10^{400}, beyond either dtype, while RMST is representable.

A final `where` is insufficient protection: autodiff can encounter an infinite or NaN derivative in an unselected expression, and batching evaluates all branches. Each branch therefore receives its own safe operands *before* the logarithm, exponential, or gamma call. Inactive gamma inputs use $k = b = x = 1$; inactive series inputs use $k = x = 1$; inactive mean inputs use $k = b = 1$. These are discarded finite calculations, not replacements for valid active inputs. Review-time mutation runs demonstrated that removing either protected exponential gives NaN gradients while final values still pass. The committed tail-gradient test now detects both mutations; mutations are not stored tests.

The public method masks invalid parameters and negative, infinite, or NaN cutoffs before calling the numerical helper, then restores NaN results. At zero, the helper substitutes before logarithms and returns the active cutoff. All three operands are promoted to a common JAX dtype. Thus a float64 cutoff can preserve a result too large for float32 even when parameters were stored in float32; at $k = 1/4$ the mean is $24b$, used by the mixed-dtype regression.

7 Derivatives and an unresolved zero-boundary question

In the smooth interior the exact identities are

$$\frac{\partial M}{\partial \tau} = S(\tau), \qquad \frac{\partial M}{\partial b} = \frac{M - \tau S(\tau)}{b},$$

$$\frac{\partial M}{\partial k} = -\int_0^{\tau} e^{-(t/b)^k} (t/b)^k \log(t/b)\, dt.$$

The implementation differentiates its piecewise expressions with JAX, not an independently integrated tangent. First shape derivatives use JAX's incomplete gamma rules. Higher derivatives involving shape are unsupported because `igamma_grad_a` has no higher rule; some orders or branches may instead succeed. Higher derivatives in scale and cutoff are supported in the smooth interior. No full shape Hessian or generic quadrature tangent is promised by this stage.

The explicit zero branch gives $M(0) = 0$, parameter derivatives 0 and first cutoff derivative 1. **Do not infer higher right-cutoff derivatives from this.** For $k = 1$, $M = b(1 - e^{-\tau/b})$, so its second right derivative at zero is $-1/b$. At $b = 2$, the current implementation returns autodiff second derivative 0 instead of $-1/2$. A float64 probe confirmed this. For $k = 2$, $b = 2$ the third right derivative is also $-1/2$, while autodiff returns 0. For $0 < k < 1$, the second right derivative diverges to $-\infty$. Thus finite derivatives of all orders at zero cannot hold throughout the promised shape range. D-028 asks whether section 8.7 intends its general higher-derivative promise to cover existing higher right derivatives at zero, or only smooth interior points beyond the explicitly specified first boundary derivatives. This report does not settle that contract or change code. The earlier invalid-entry-own-tangent question S-1 also remains open.

8 What is verified, and what remains for approval

The committed tests use independent 60-digit mpmath quadrature at six shapes from 0.2 to 50, seven cutoff ratios from 0.01 to 10^5, and scale factors 10^{-4}, 1 and 10^4, with eager/JIT/vmap checks in both dtype processes. They cover zero/invalid cases, shape-one identities, mixed dtype promotion, interior first derivatives, a supported Hessian, the shape-Hessian exception, and finite gradients at both power stress tails. The final focused RMST suite reported 21 passed per dtype. The unchanged production code passed the fresh-wheel through-A10 regression gate before the test-only tail-gradient fix.

Independent review round 2 returned `merge` for the implementation step. Its extra value probes reported worst relative errors about 4.0×10^{-15} (float64) and 2.0×10^{-6} (float32). These are measured samples, not uniform proofs. Its direct gate had five wheel-test setup errors from an unset wheel variable; permissions blocked its fresh-wheel wrapper. Our fresh-wheel gate included those tests. The complete measured first-derivative sweep and derivative tolerance factors remain RMST.3 work. D-028 must be considered before approving boundary claims.

9 Should RMST have a grid method?

There is no mathematical obstacle: `rmst_grid(t)` could be a thin adapter that adds one trailing singleton per batch axis. However, spec 8.7 explicitly says there is no `rmst_grid` in this staged release. Current usage for a batch of shape `(S, 2)` is:

```
values = dist.rmst(tau_grid[:, None, None])  # (m, S, 2)
```

I recommend keeping that staged surface for this step. A grid convenience method could improve discoverability and consistency later, but needs an explicit API/spec decision; it would add no numerical capability or approximation. A bare `(m,)` cutoff vector follows ordinary trailing broadcasting and can pair with a batch axis instead of producing a Cartesian grid.

10 Sources and reproducible PDF build

Full source commit: `b3340749469e1513938a0cd5140f45305ab0d4bb`.

Local authority: spec sections 7.2, 8.7 and 12.1, the code and RMST tests at the source commit above, and `review/slices/RMST.2.md`. DLMF links accompany the identities; the error bounds are derived here.

From the repository root, with the existing Pandoc and XeLaTeX installations:

```
pandoc reports/weibull-rmst-math.md --standalone --number-sections \
  --pdf-engine=xelatex -o output/pdf/weibull-rmst-math.pdf
```